

Template parameter deduction for constructors (Rev. 3)

Summary

This paper proposes extending template parameter deduction for functions to constructors of template classes and incorporates feedback from the EWG review of [N4471](#).

Currently, if we want to construct template classes, we need to specify the template parameters. For example, [N4498](#) on Variadic Lock Guards gives an example of acquiring a `lock_guard` on two mutexes inside an `operator=` to properly lock both the source and destination of the assignment. Expanding typedefs, the locks are acquired by the following statement (See the paper for details and rationale).

```
std::lock_guard<std::shared_timed_mutex, std::shared_lock<std::shared_timed_mutex>> lck(mut_, r1);
```

Having to specify the template parameters adds nothing but complexity! If constructors could deduce their template parameters "like we expect from other functions and methods," then the following vastly simpler and more intuitive code could have been used instead.

```
auto lock = std::lock_guard(mut_, r1);
```

Let's not think that this problem could have been avoided if only `lock_guard` had a "make function," `lock_guard` is not moveable and therefore cannot have a make function. Indeed, LEWG explicitly declined to make it moveable before advancing the paper to LWG. As we will see below, even for moveable class, "make functions" have their own problems.

The sections below first spell out the problem in more detail and then makes precise what "like we expect from other functions and methods" means in this context.

The problem

To simplify the examples below, suppose the following definitions are in place.

```
vector<int> vi1 = { 0, 1, 1, 2, 3, 5, 8 };  
vector<int> vi2;  
template<class Func>  
std::mutex m;  
unique_lock<std::mutex> ul(m, std::defer_lock);  
class Foo() {  
public:  
    Foo(Func f) : func(f) {}  
    void operator()(int i) {  
        os << "Calling with " << i << endl;  
        f(i);  
    }  
private:  
    Func func;  
    mutex mtx;  
};
```

In the current standard, the following objects would be constructed as shown

```
pair<int, double> p(2, 4.5);  
auto t = make_tuple(4, 3, 2.5);  
copy_n(vi1, 3, back_inserter(vi2));  
// Virtually impossible to pass a lambda to a template class' constructor  
for_each(vi2.begin(), vi2.end(), Foo<??>([&](int i) { ...}));
```

```
lock_guard<std::mutex> lck(foo.mtx);
lock_guard<std::mutex, std::unique_lock<std::mutex>> lck2(foo.mtx, ul); // Notation from N4470
auto hasher = [](X const & x) -> size_t { /* ... */ };
unordered_map<X, int, decltype(hasher)> ximap(10, hasher);
```

There are several problems with the above code:

- Creating “make functions” like `make_tuple` is confusing, artificial, extra boilerplate, and inconsistent with how non-template classes are constructed.
- Since the standard library doesn't follow any consistent convention for make functions, users have to scurry through documentation to find they need to use `make_tuple` for tuples and `back_inserter` for `back_insert_iterators`. Of course their own template classes might not be as consistent or thoroughly-documented as the standard.
- Specifying template parameters as in `pair<int, double>(2, 4.5)` should be unnecessary since they can be inferred from the type of the arguments, as is usual with template functions (this is the reason the standard provides make functions for many template classes in the first place!).
- A make function may do more than just deduce constructor template parameters. Unless a detailed study of the documentation is made (which usually only happens when debugging weird unexpected behavior...), subtle changes in semantics may occur. Committee members and other C++ experts are invited to see if they can tell which of the above make functions simply do the “obvious” deduction of template parameters.
- If we don't have a make function, we may not be able to create class objects at all as indicated by the `???` in the code above.
- Make functions can't be used with classes that aren't movable like `std::lock_guard`.
- The useful technique of replacing a large function with a class by organizing its code into methods to reduce cyclomatic complexity can't be used for template functions

If we allowed the compiler to deduce the template parameters for constructors of template classes, we could replace the above with:

```
pair p(2, 4.5);
tuple t(4, 3, 2.5);
copy_n(vi1, 3, back_inserter(vi2));
for_each(vi.begin(), vi.end(), Foo([&](int i) { ...})); // Now easy instead of virtually impossible
auto lck = lock_guard(foo.mtx);
lock_guard lck2(foo.mtx, ul);
unordered_map<X, int> ximap(10, [](X const & x) -> size_t { /* ... */ });
```

We believe this is more consistent and simpler for both users and writers of template classes, especially for user-defined classes that might not have carefully designed and documented make functions like `pair`, `tuple`, and `back_inserter`.

The Solution

We propose two techniques

- [Auto-deduction of existing constructors](#)
- [Canonical factory functions](#)

These techniques work well together as will be explained below. They can also be adopted separately (E.g., as a result of the discussion on [Code Compatibility](#) below).

Auto-deduction of existing constructors

We propose to allow a template name referring to a class template as a simple-type-specifier in two contexts:

- Functional-notation simple type conversions (`[expr.type.conv]`), and
- Simple-declarations of the form *"decl-specifier-seq id-expression initializer"*.

In the case of a function-notation type conversion (e.g., `"tuple(1, 2.0, false)"`) or a direct parenthesized or braced initialization, the initialization is resolved as follows. First, constructors and constructor templates declared in the named template are enumerated. Let C_i be such a constructor or constructor template; together they form an overload set. A parallel overload set F of function templates is then created as follows: For each C_i a function template is constructed with template parameters that include both those of the named class template and if C_i is a constructor template, those of that template (default arguments are included too) -- the function parameters are the constructor parameters, and the return type is `void`. Deduction and overload resolution is then performed for a synthesized call to F with the parenthesized or braced expressions used as arguments. If that call doesn't yield a "best viable function", the program is ill-formed. Otherwise, the template name is treated as the class-name that is obtained from the named class template with the deduced arguments corresponding to that template's parameters.

Let's look at an example:

```
template<typename T> struct S {
    template<typename U> struct N {
        N(T);
        N(T, U);
        template<typename V> N(V, U);
    };
};

S<int>::N x{2.0, 1};
```

In this example, "S<int>::N" in the declaration of `x` is missing template arguments, so the approach above kicks in. Template arguments can only be left out this way from the "type" of the declaration, but not from any name qualifiers used in naming the template; i.e., we couldn't replace "S<int>::N" by just "S::N" using some sort of additional level of deduction. To deduce the initialized type, the compiler now creates an overload set as follows:

```
template<typename U> void F(S<int>::N<U> const&);
template<typename U> void F(S<int>::N<U> &&);
template<typename U> void F(int);
template<typename U> void F(int, U);
template<typename U, typename V> void F(V, U);
```

(The first two candidates correspond to the implicitly-declared copy and move constructors. Note that template parameter `T` is already known to be `int` and is not a template parameter in the synthesized overload set.) Then the compiler performs overload resolution for a call "`F(2.0, 1)`" which in this case finds a unique best candidate in the last synthesized function with `U = int` and `V = double`. The initialization is therefore treated as "`S<int>::N<int> x{2.0, 1};`;"

Note that after the deduction process described above the initialization may still end up being ill-formed. For example, a selected constructor might be inaccessible or deleted, or the selected template instance might have been specialized or partially specialized in such a way that the candidate constructors will not match the initializer.

The case of a simple-declaration with copy-initialization syntax is treated similarly to the approach described above, except that explicit constructors and constructor templates are ignored, and the initializer expression is used as the single call argument during the deduction process.

Canonical factory functions

While the above procedure generates many useful deducible constructors, some constructors that we would like to be deducible are not. For example, one could imagine a function `make_vector` defined as follows:

```
template<typename Iter>
vector<Iter::value_type> make_vec(Iter b, Iter e) {
    return vector<Iter::value_type>(b, e);
}
```

Although there is no constructor in `vector` from which we can deduce the type of the vector from two iterators, one would like to be able to deduce the type of the vector from the value type of the two iterators. For example, some implementations of the STL define their `value_type` typedef as follows

```
template<typename T, typename Alloc=std::allocator<T>>
struct vector {
    struct iterator {
        typedef T value_type;
        /* ... */
    };
    typedef iterator::value_type value_type;
    /* ... */
};
```

The detour through `vector<T>::iterator` keeps us from deducing that `T` is `char` in a constructor call like `vector(5, 'c')`. We would certainly like constructors like that to work.

We suggest a notation to allow constructors to specify their template parameters by either explicitly declaring the signatures for

any further needed constructor deductions outside the class

```
template<typename T, typename Alloc = std::allocator<T>> struct vector {  
    /* ... */  
};  
template<typename Iter> vector(Iter b, Iter e) -> vector<typename iterator_traits<Iter>::value_type>
```

In effect, this allows any pure factory function to be specified with any deduction rules that are specifiable by any function with a standard first-class name and no boilerplate code in the body. It also allows us to suppress a standard deduction from the above process via “= delete;”

Alternatively, we could use “declaration notation”

```
template<typename Iter> vector<typename iterator_traits<Iter>::value_type> vector(Iter b, Iter e);  
template<typename Iter> auto vector(Iter b, Iter e) -> vector<typename iterator_traits<Iter>::value_type>;
```

Note that it is only necessary to declare canonical factory functions. Giving a definition is not allowed as they just construct their return type from their arguments according to normal rules. This restriction makes it instantly visible that any use of a constructor as a factory simply constructs the class and does not have any obscure semantics.

A note on injected class names

The focus on this paper is on simplifying the interface of a class for its clients. Within a class, one may need to explicitly specify the parameters as before due to the injected class name:

```
template<typename T> struct X {  
    template<typename Iter>  
    X(Iter b, Iter e) { /* ... */ }  
  
    template<typename Iter>  
    auto foo(Iter b, Iter e) {  
        return X(b, e); // X<U> to avoid breaking change  
    }  
  
    template<typename Iter>  
    auto bar(Iter b, Iter e) {  
        return X<Iter::value_type>(b, e); // Must specify what we want  
    }  
};
```

Code compatibility

While we cannot say whether it is a problem in practice, we should point out a scenario where auto-deduction can break compatibility.

Suppose I produce a library and I'm under license to preserve source compatibility across all 1.x upgrades, and I have this class template in version 1.0:

```
template struct X {  
    X(T);  
};
```

... and in version 1.1 I rewrite it as this:

```
template struct X {  
    struct iterator { typedef T type; };  
    X(typename iterator::type);  
};
```

If one of my users upgrades to C++17, with this change in the language, I am no longer complying with the terms of my licensing. Likewise, if this language change happens between me releasing 1.0 and 1.1, I can no longer release version 1.1 because it might break some of my existing customers.

The point is: current code does not express any intent about whether class template parameters are deducible, based on whether

they use the version 1.0 code or the version 1.1 code. But this change makes that implicit property into part of the de facto interface of the code.

So why include auto-deduction at all in this proposal as canonical factory functions could be used to make explicit all of the auto-deduced constructors? Basically, having to manually specify what is obviously expected has an insidious cost as any (honest) Java programmer can tell you so we would like to discuss with the committee whether this will be an actual problem in practice before jettisoning a useful feature and make programmers manually insert the “obvious” canonical factory functions for all of their existing classes.